

Bitte beantworten Sie die Fragen in den dafür vorgesehenen Feldern. Wenn Sie keinen Platz mehr für eine Antwort haben, fahren Sie bitte auf der Rückseite fort und geben an wo es weiter geht.
Abgesehen von Internetfähigen Geräten sind alle Hilfsmittel zugelassen.
Schreiben Sie Ihre Klausur bitte nicht mit Bleistift und lassen Sie die Klausur bitte geheftet.
Sie haben 120 Minuten Zeit. Mit 53 Punkten gilt die Klausur als bestanden.

Vor- und Nachname (in Druckbuchstaben): _____

Matrikelnummer und Studiengang: _____

Aufgabe Nr.:	Punktzahl:	Davon erreicht:
1	15	
2	7	
3	7	
4	7	
5	13	
6	14	
7	11	
8	8	
9	13	
10	14	
11	11	
Summe	120	

Note und Unterschrift des Erstprüfers: _____

Unterschrift des Zweitprüfers: _____

Lernraum I

1. (15 Punkte) Sie arbeiten an einer Anwendung, die eine Sammlung von Büchern verwalten soll. Jedes Buch hat eine eindeutige ISBN-Nummer, einen Titel, einen Autor und ein Veröffentlichungsdatum. Die Anwendung soll in der Lage sein, die folgenden vier Operationen effizient auszuführen:
 1. Einfügen eines neuen Buches mit allen seinen Daten.
 2. Suchen eines Buches anhand der ISBN.
 3. Aktualisieren des Autors oder des Veröffentlichungsdatums eines Buches.
 4. Löschen eines Buches aus der Sammlung.

Wählen Sie einen geeigneten abstrakten Datentyp und eine Datenstruktur, um die Buchdaten effizient zu verwalten und die oben genannten Operationen optimal durchzuführen. Begründen Sie Ihre Wahl und erläutern Sie, warum dieser abstrakte Datentyp und diese Datenstruktur geeignet sind. Geben Sie zusätzlich die erwarteten Laufzeiten für die vier oben genannten Operationen in der O-Notation an.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

2. (7 Punkte) Was ist die asymptotische Laufzeit der folgenden Funktion f? Geben Sie eine Θ -Schranke an. Begründen Sie Ihre Antwort kurz.

```
void f(int n) {
    int i = 1;
    int j;
    while(i < n*n*n*n) {
        j = n;
        while (j > 2) {
            j /= 2;
        }
        i += n*n;
    }
}
```

.....

.....

.....

.....

.....

.....

.....

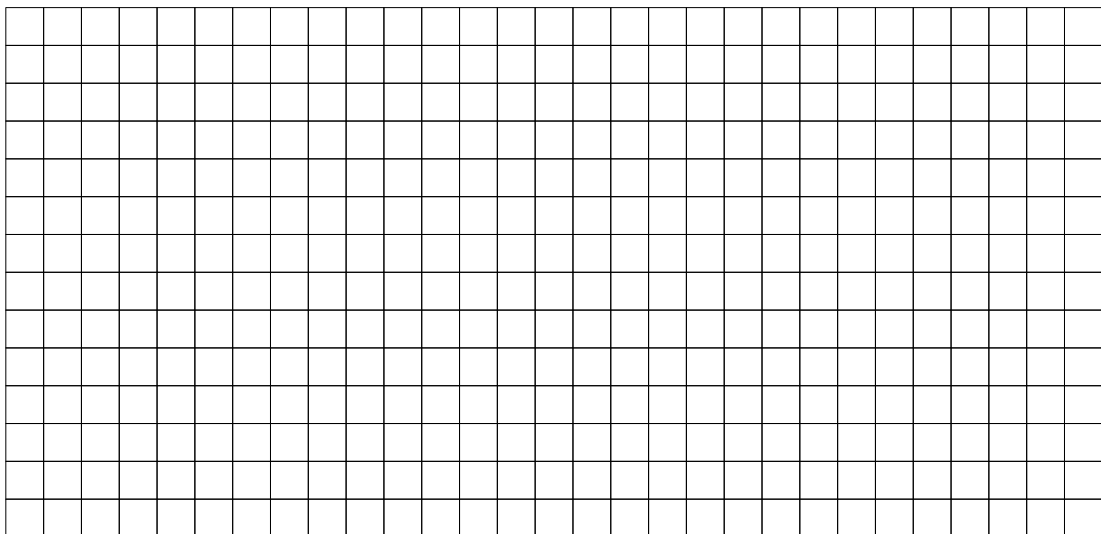
.....

.....

.....

3. (7 Punkte) Bestimmen Sie die Θ -Notation für die geschlossene Form der folgenden Rekursion.

$$T(n) = \begin{cases} 100 & \text{falls } n \leq 5 \\ 9T\left(\frac{n}{3}\right) + 3 \cdot n^2 + \log n & \text{sonst} \end{cases}$$



4. (7 Punkte) Geben Sie für die folgende Funktion `getsum` die Θ -Notation für deren Laufzeit im Worst Case sowie die Θ -Notation für deren Laufzeit im Best Case in Abhängigkeit von n , der Größe des eindimensionalen Eingabe-Arrays `arr`, an. Welche Eigenschaft hat das Eingabe-Array `arr` im Worst Case und welche Eigenschaft im Best Case? Sie müssen nicht begründen wie Sie auf die Θ -Notation kommen.

```
def getsum(arr):
    n = len(arr)

    if n % 2 == 0:
        return 0

    Sum = 0
    for i in range(n):
        Sum += arr[i]

    return Sum

if __name__ == '__main__':
    arr1 = [1, 2, 3, 4] # Declaring an array

    # Function call
    print(getsum(arr1))
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Lernraum II

5. Hashing mit offener Adressierung: Zur Speicherung von Zeichenketten in einer Hashtabelle der Größe 11 soll folgende Hashfunktion `hash_code` verwendet werden. Bei Kollisionen soll lineare Sondierung genutzt werden. Hinweis: Der ASCII-Wert des Zeichens “a” beträgt 97, “b” entsprechend 98, etc. Dieser Wert wird von der Funktion `ord` zurückgegeben.

```
def hash_code(input_string):
    output = 0
    for c in input_string: # for each character c in the input_string
        output += ord(c) - 90
    return output % 11
```

- (a) (8 Punkte) Tragen Sie die folgenden Zeichenketten in der angegebenen Reihenfolge in eine anfangs leere Hashtabelle ein. Tragen Sie dazu die Zeichenketten in die korrekten Indizes der gegebenen Hashtabelle ein. Bitte geben Sie den Rechenweg an.

Zeichenketten: "abc", "ac", "bc", "ca"

[illegible]This image shows a full page of blank graph paper. The grid consists of small, equal-sized squares formed by thin black lines. There are 20 columns and 20 rows of squares, creating a total of 400 square units. The grid is perfectly aligned and covers the entire area of the page.

- (b) (5 Punkte) Geben Sie an, wie viele Zugriffe auf die Hashtabelle für die erfolglose Suche nach dem Schlüssel “ccc” benötigt werden. Auf welche Indizes der Hashtabelle wird zugegriffen?

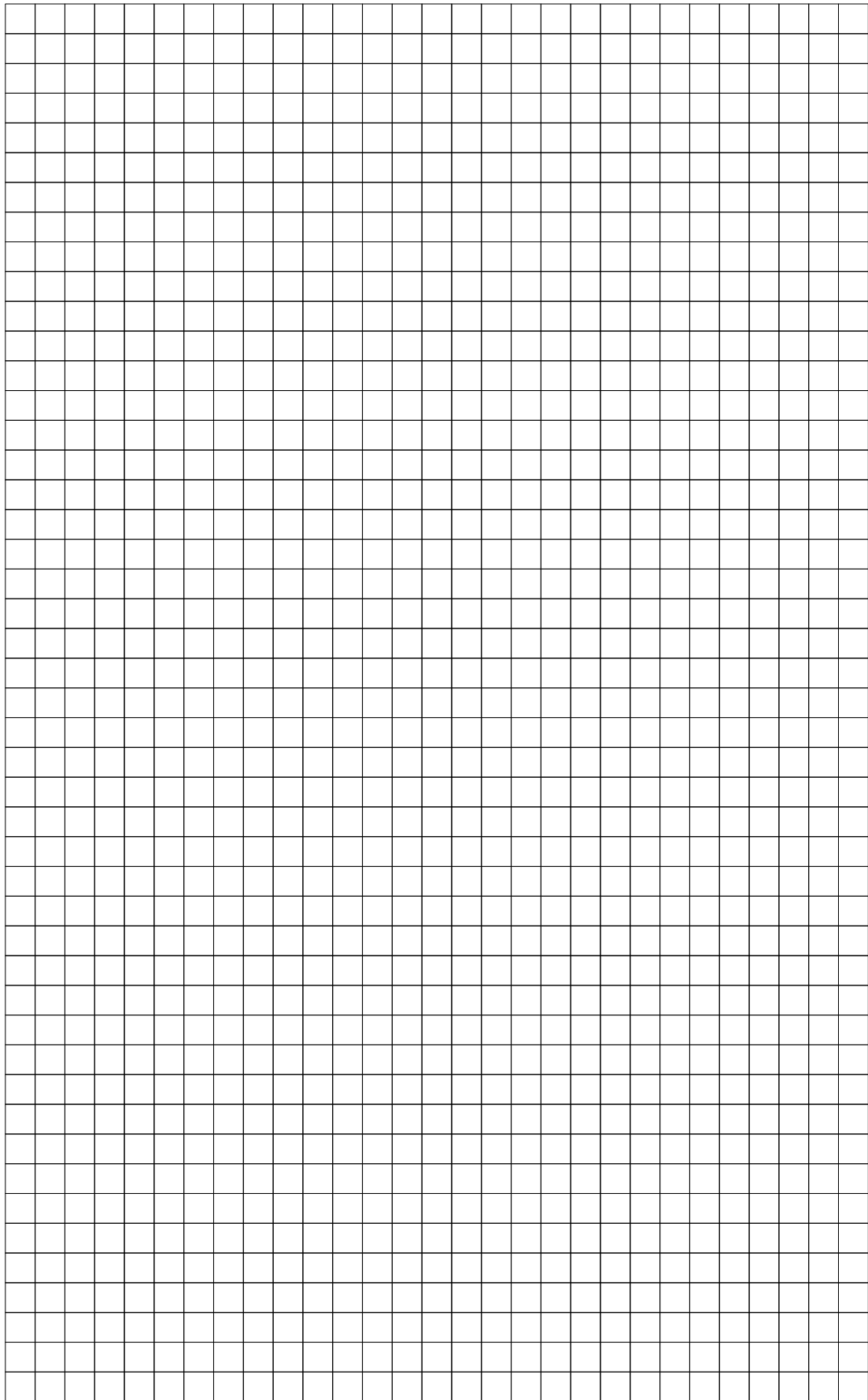
[illegible]

6. (14 Punkte) Fügen Sie die folgende Zahlenfolge in einen **Max**-Heap ein, indem Sie den buildHeap-Algorithmus von Floyd verwenden. Markieren Sie Ihre finale Lösung.

[10, 7, 15, 17, 12, 20, 6, 32]

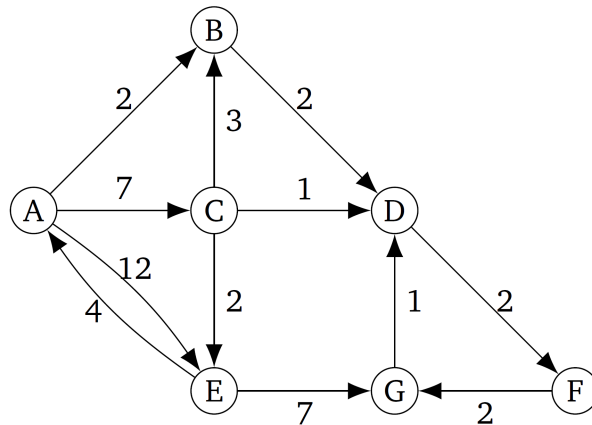
This image shows a full page of blank graph paper. The grid consists of small, uniform squares formed by thin gray lines. There are no margins, text, or other markings on the page.

Fortsetzung der Rechenkästchen auf der nächsten Seite



Lernraum III

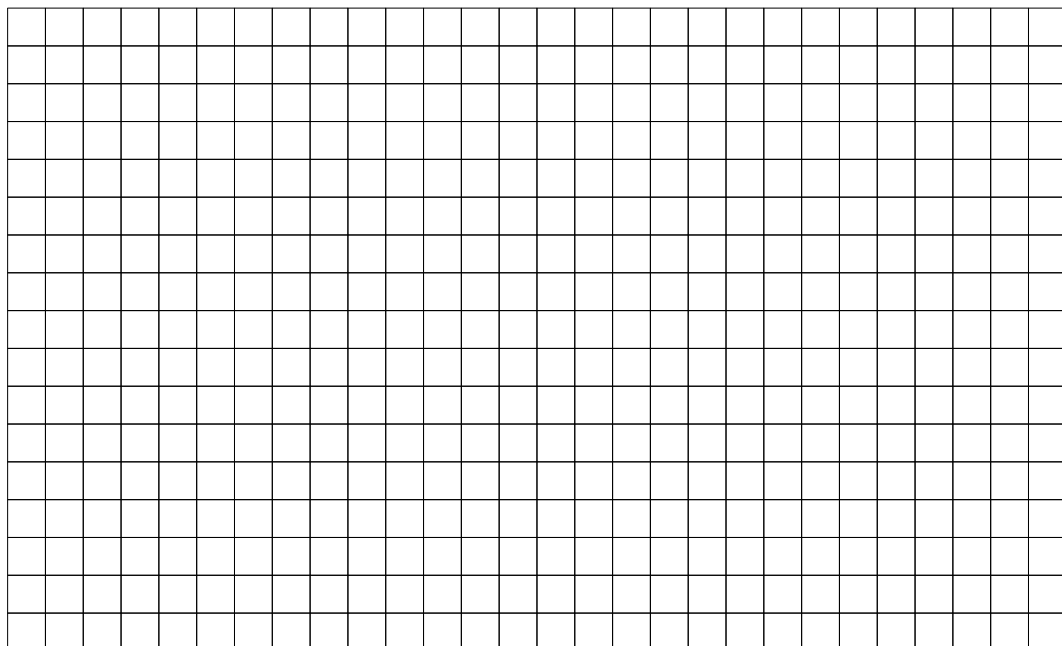
7. Kürzeste Pfade in einem gewichteten Graphen.



- (a) (8 Punkte) Bestimmen Sie den kürzesten Pfad vom Startknoten A aus zu allen anderen Knoten des obigen Graphen. Sie können dazu folgende Tabelle nutzen. Die kürzesten Abstände vom Knoten A aus sollen am Ende in der Spalte Distanz stehen.

Knoten	Distanz	Vorgänger	Verarbeitet?
A			
B			
C			
D			
E			
F			
G			

- (b) (3 Punkte) Zeichnen Sie den Baum kürzester Pfade von Startknoten A aus.



8. (8 Punkte) Nach einem schweren Orkan sind einige Orte von der Außenwelt abgeschnitten, da einige Straßen durch umgestürzte Bäume etc. nicht passierbar sind. Ihre Aufgabe ist, möglichst schnell Hilfsgüter zu den abgeschnittenen Orten zu bringen. Sie möchten die Orte über einen Hubschrauber versorgen. Mit welchem Algorithmus können Sie bestimmen welche Flugroute der Hubschrauber nehmen sollte, damit alle Orte in insgesamt kürzester Zeit zu erreichen sind? Wie können Sie bestimmen, ob der Hubschrauber alle Orte mit einer Tankladung erreichen kann (den Flug zurück zum Startpunkt können Sie ignorieren)?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Lernraum IV

9. Sie wollen im Sommer eine Tour zwischen den Geburtsstätten der wichtigsten deutschen Musiker machen. Sie möchten die Tour mit dem Kanu durchführen. Eine Karte von Deutschland mit allen Gewässern und Flüssen sei Ihnen gegeben.

Um sich auf die Musiker einzustimmen, möchten Sie während der Kanutour deren Lieder in chronologischer Reihenfolge hören von alt nach neu. Nehmen Sie an, dass Sie die Musiksammlung als Tabelle abspeichern, die Sie als Array implementieren. Als Schlüssel der Schlüssel-Wert-Paare wählen Sie Nach- und Vorname des Künstlers inkl. Songtitel. Werte sind bspw. Künstlername, Songtitel, Erscheinungsdatum, Pfad zur MP3-Datei, ...

- (a) (9 Punkte) Welchen Sortieralgorithmus könnten Sie idealerweise nutzen, um die Lieder wie gewünscht zu sortieren? Welche Eigenschaften muss der Algorithmus haben? Bedenken Sie, dass Sie auf dem Kanu nur einen sehr kleinen Rechner dabei haben. Begründen Sie Ihre Antwort. Diese Frage hat mehrere korrekte Antworten.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

- (b) (4 Punkte) Wählen Sie eine Datenstruktur mit der Sie die Reiseroute planen können. Wie werden Geburtsstädte, Wasserwege, Entfernungen zwischen den Geburtsstädten in der Datenstruktur gespeichert/modelliert?

.....

.....

.....

.....

.....

.....

.....

.....

10. (14 Punkte) Das Problem der längsten gemeinsamen Teilsequenz (longest common subsequence - LCS) besteht darin, die längste Teilsequenz zu finden, die in gegebenen zwei Sequenzen in derselben Reihenfolge vorhanden ist, d.h. die längste Sequenz zu finden, die aus der ersten Originalsequenz durch Löschen einiger Elemente und aus der zweiten Originalsequenz durch Löschen einiger Elemente erhalten werden kann.

Beispiel: Gegeben die beiden Sequenzen

X: CBAD

Y: CAB

Die Länge der LCS beträgt 2, da die LCS lautet: CA.

Die Brute-Force Lösung lautet:

```
# Funktion zum Ermitteln der Länge der längsten gemeinsamen Teilfolge von
# Sequenzen 'X[0m-1]' und 'Y[0n-1]'
def LCSLength(X, Y, m, n):

    # kehrt zurück, wenn das Ende einer der beiden Sequenzen erreicht ist
    if m == 0 or n == 0:
        return 0

    #, wenn das letzte Zeichen von 'X' und 'Y' übereinstimmt
    if X[m - 1] == Y[n - 1]:
        return LCSLength(X, Y, m - 1, n - 1) + 1

    # andernfalls, wenn das letzte Zeichen von 'X' und 'Y' nicht übereinstimmt
    return max(LCSLength(X, Y, m, n - 1), LCSLength(X, Y, m - 1, n))

if __name__ == '__main__':

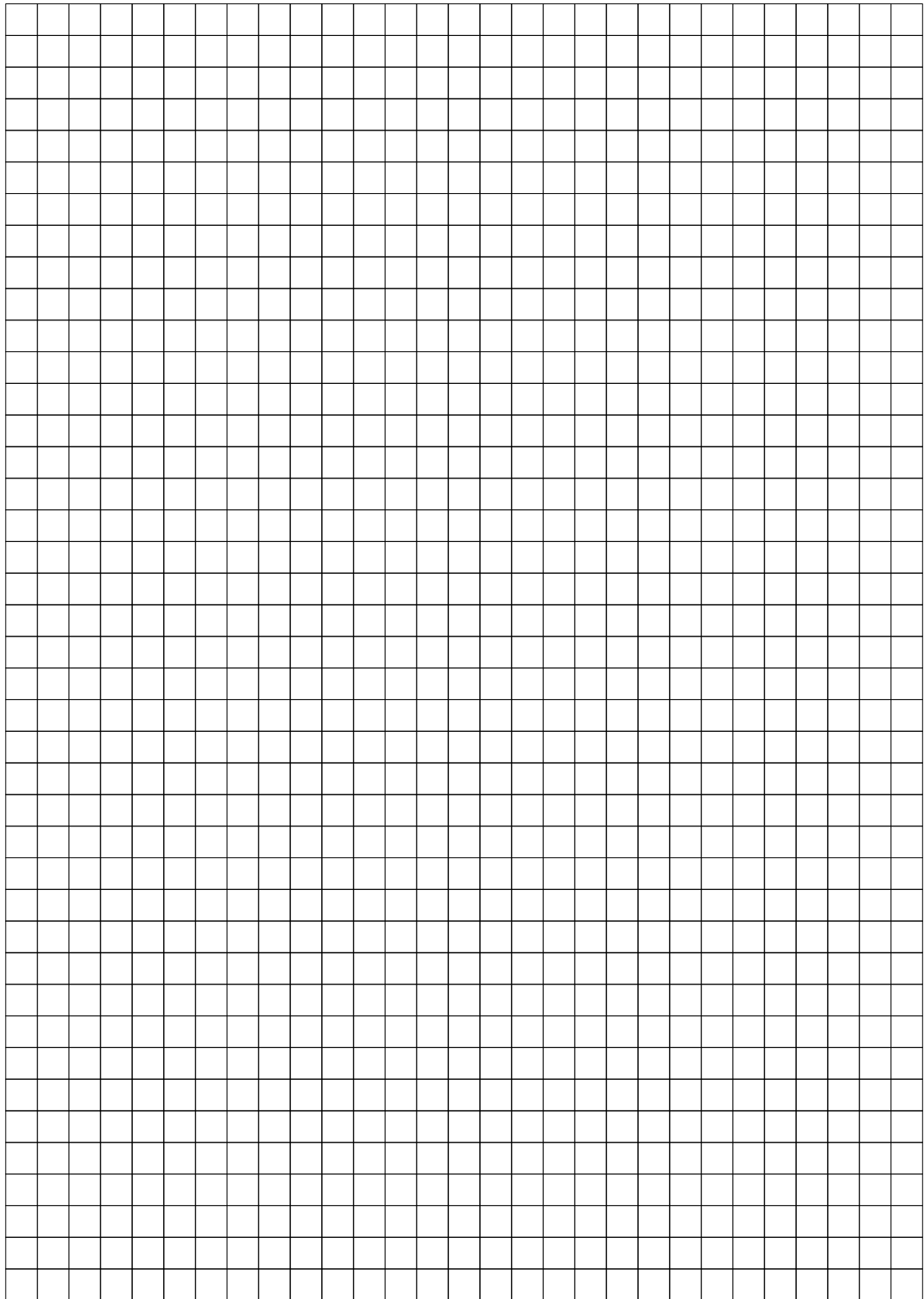
    X = 'CBAD'
    Y = 'CAB'

    print('The length of the LCS is', LCSLength(X, Y, len(X), len(Y)))
```

Zeigen Sie, dass dieser Code ineffizient ist, da die Funktion `LCSLength` häufig mit identischen Argumenten aufgerufen wird. Zeichnen Sie dazu den Rekursionsbaum der Methode `LCSLength` für das im Code aufgerufene Beispiel bis zur **maximalen Höhe von 3**. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente `m` und `n` einzuzichnen. Der Wurzelknoten des Rekursionsbaums sieht also so aus:

4, 3

Rechenkästchen folgen auf der nächsten Seite.



11. (11 Punkte) Schauen Sie sich die beiden Funktionen f1 und f2 an.

```
import random

f1(n):
    array = [k for k in range(1, n+1)]

    order_1 = [k for k in range(0, n)]
    order_2 = [k for k in range(0, n)]
    # after this call the digits in order_2 are in random order
    random.shuffle(order_2)

    sum = 0
    for i in range(n):
        sum += array[order_2[i]]

    print(sum)

f2(n):
    array = [k for k in range(1, n+1)]

    order_1 = [k for k in range(0, n)]
    order_2 = [k for k in range(0, n)]
    # after this call the digits in order_2 are in random order
    random.shuffle(order_2)

    sum = 0
    for i in range(n):
        sum += array[order_1[i]]

    print(sum)
```

Welche Funktion, f1 oder f2, wird in der Praxis für **sehr** große n (bspw. $n \geq 10^6$) schneller laufen? Begründen Sie Ihre Antwort und erläutern Sie eventuell genutzte Fachbegriffe.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....